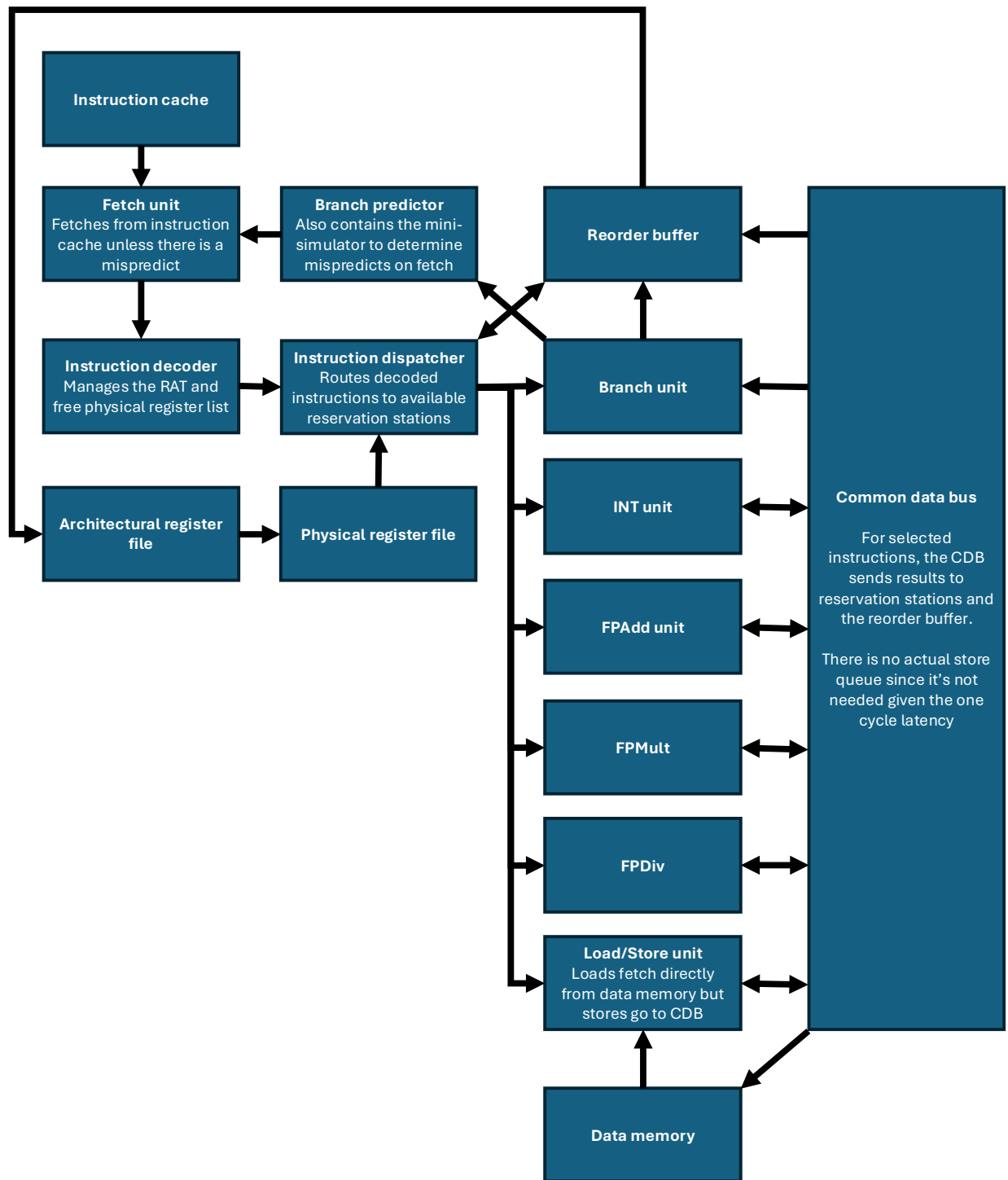




To keep the project organized, I made a separate class for most of the physical components of the CPU. The instruction and data cache don't have their own files, but I included them in the figure since they represent major physical components of the system. The layout is pretty similar to what is shown on the slides, and the arrows point in the direction of data flow. I also made an abstract base class for the functional units not shown in the figure. To make cross-component communication easier, I kept all the variables on the simulator public, and passed in reference to the simulator to the components as needed.

To split the components into pipeline phases (roughly):

Fetch – instruction cache, fetch unit, branch predictor

Decode – instruction decoder

Dispatch – instruction dispatcher, architectural register file

Execute – load/store unit, INT unit, FPAdd unit, FPMult unit, FPDIV unit, branch unit

Commit – Common data bus, reorder buffer, architectural register file

Here's a list of data objects I made to store and pass information between components:

- Branch history table entry
- Branch target buffer entry
- Instruction (architectural registers)
- Decoded instructions (physical registers)
- Reorder buffer entry
- Reservation station

I separated out these two data types from simulator to make them easier to reference from other files

- Architectural register
- RegisterFileEntry